# Media Recommendation Algorithm — Mathematical Formulation

This document walks through the content-based algorithm behind suggesting unwatched movies and TV shows to a user, built from their rating history, cast and crew overlap, how popular a title is, what their friends thought of it, and how recently it came out. The scoring itself is entirely deterministic and needs no AI to run at all; Gemini is introduced afterward, in a deliberately limited way, to lightly refine the shortlist and add a short explanation to each result (see Section 7).

## 1. Notation

- U = the target user.
- R(U) = set of media the user has rated.
- $R^+(U) = \{ m \in R(U) : rating(U, m) \geq 4 \}$, the user's highly-rated media (their “taste profile”).
- C = a candidate media item (not yet watched by U).
- genres(x), cast(x) = the set of genres / cast+crew members for media x.
- F(U) = the subset of U's friends who have rated C.
- currentYear = the current calendar year; releaseYear(C) = C's release year.

## 2. Build the User Taste Profile

Before scoring any candidates, compute the union of genres and cast/crew across the user's highly-rated media:

$$genreProfile(U) = \bigcup genres(m) \quad for\ all\ m \in R^+(U)$$

$$castProfile(U) = \bigcup cast(m) \quad for\ all\ m \in R^+(U)$$

## 3. Per-Candidate Sub-Scores

For each candidate C, compute five sub-scores, each normalized to the range [0, 1]:

Genre overlap score (weight 0.40):

$$G(C) = |\, genres(C) \cap genreProfile(U) \,| \div |\, genres(C) \,|$$

Cast/crew overlap score (weight 0.20):

$$CastScore(C) = |\, cast(C) \cap castProfile(U) \,| \div |\, cast(C) \,|$$

Popularity score (weight 0.15) — TMDB average rating normalized from a 0–10 scale:

$$P(C) = avgRating(C) \div 10$$

Friends' rating score (weight 0.15) — average of friends' star ratings (1–5), normalized:

$$FriendScore(C) = 0, \text{ if } F(U) = \emptyset$$

$$FriendScore(C) = \left( \Sigma\, rating(f, C) \text{ for } f \in F(U) \right) \div \left( |F(U)| \times 5 \right), \quad otherwise$$

Recency score (weight 0.10) — linear decay over a K-year window (K = 15):

$$Recency(C) = max(\ 0,\ 1 - (currentYear - releaseYear(C)) \div K\ )$$

### 4. Final Weighted Score

$$Score(C) = 0.40 \cdot G(C) + 0.20 \cdot CastScore(C) + 0.15 \cdot P(C) + 0.15 \cdot FriendScore(C) + 0.10 \cdot Recency(C)$$

The five weights sum to 1.0. They are configurable constants rather than derived values, and are best tuned after testing against real user data — for instance, lowering the genre weight if recommendations start to feel repetitive.

### 5. Algorithm Steps

1. Filter the user's reviews to build $R^+(U)$ (ratings ≥ 4 stars).
2. Build genreProfile(U) and castProfile(U) from $R^+(U)$.
3. Fetch a candidate pool of unwatched media (e.g., trending/popular titles from TMDB, or titles sharing at least one genre with genreProfile(U), to keep the search space manageable).
4. For each candidate C, compute G(C), CastScore(C), P(C), FriendScore(C), and Recency(C).
5. Compute Score(C) using the weighted sum above.
6. Sort all candidates by Score(C) descending.
7. Return the top N candidates (e.g., N = 5–10) as the recommendation list.

### 6. Worked Example

Suppose user U has rated:

- Movie A — Action/Sci-Fi, 5 stars, cast {Actor1, Actor2}, released 2020.
- Movie B — Drama, 3 stars (excluded — below the 4-star threshold).
- Movie C — Sci-Fi/Thriller, 4 stars, cast {Actor3}, released 2018.

This gives $R^+(U)$ = {A, C}, so:

$$genreProfile(U) = \{Action, Sci\text{-}Fi, Thriller\}$$

$$castProfile(U) = \{Actor1, Actor2, Actor3\}$$

Candidate D: genres {Sci-Fi, Adventure}, cast {Actor2, Actor4}, TMDB rating 7.8/10, released 2024. Two friends rated D at 5 and 4 stars.

| Sub-score | Calculation | Value |
|---|---|---|
| G(D) | $\lvert\{Sci\text{-}Fi, Adventure\} \cap \{Action, Sci\text{-}Fi, Thriller\}\rvert \div 2 = 1 \div 2$ | 0.50 |

| | | |
|---|---|---|
| CastScore(D) | \|{Actor2,Actor4} ∩ {Actor1,Actor2,Actor3}\| ÷ 2 = 1 ÷ 2 | 0.50 |
| P(D) | 7.8 ÷ 10 | 0.78 |
| FriendScore(D) | (5 + 4) ÷ (2 × 5) | 0.90 |
| Recency(D) | 1 − (2026 − 2024) ÷ 15 | 0.867 |

$$Score(D) = 0.40(0.50) + 0.20(0.50) + 0.15(0.78) + 0.15(0.90) + 0.10(0.867)$$

$$Score(D) = 0.20 + 0.10 + 0.117 + 0.135 + 0.0867 = 0.639$$

Candidate D would receive a final recommendation score of approximately 0.64, and would be ranked against all other candidates using the same formula — the highest-scoring N titles become the user's recommendation list.

## 7. Where AI Fits In

Everything through Section 6 stays exactly as described: a fully deterministic score built from genre overlap, cast overlap, popularity, friends' ratings, and recency. Gemini is layered on top of that in two limited ways, neither of which lets it drive the recommendation on its own.

First, only the top candidates by deterministic score, a shortlist slightly larger than what's actually shown, are ever passed to the AI at all; anything outside that shortlist stays invisible to it no matter what it might say. Second, for each shortlisted candidate the AI is asked for a single small number: a nudge to that candidate's score, capped at plus or minus 0.05, meant to capture nuance like tone or theme that the fixed formula can't see from genre and cast overlap alone. That nudge is added to the deterministic score, the shortlist is re-sorted, and only then are the final results picked and given a one-sentence explanation.

Because the adjustment is clamped in two places, once by the interactor and again defensively inside the Gemini implementation, a single AI response, however strange, can never flip a low-scoring title above a much stronger one; at most it reorders titles that were already close together. And if the API key is missing, the call fails, or Gemini returns something unparsable, the adjustment silently defaults to 0 and the deterministic ranking is what the user sees, unchanged.